

EVALUACIÓN CONTACTO CON EL DOCENTE

Miguel Ángel Suarez Prada

Docente:

Estefania Vanessa Heredia Jiménez

Universidad Internacional del Ecuador

Escuela de Ciencias de la Computación

Ingeniería en Software

Quito, Ecuador junio 2026

REINVENTEMOS
EL FUTURO

Tabla de contenido

Contenido

1.Introducción	3
2. Descripción del problema	4
3.Relación con los contenidos de la asignatura	4
4. Explicación del sistema desarrollado	7
a. Diagrama lógico del juego.....	10
b.Diagrama de funcionamiento en código	11
5. Reflexión sobre el impacto de la tecnología	13

REINVENTEMOS
EL FUTURO

1.Introducción

Para un ingeniero de software, dominar la programación es clave para estructurar soluciones lógicas y algorítmicas frente a desafíos cotidianos. Tomando esto en cuenta, el objetivo de esta iniciativa es desarrollar una herramienta interactiva centrada mediante el juego del Ahorcado, empleando Python como lenguaje de desarrollo principal. Este proyecto se enfoca en la enseñanza en la gestión de facturación dentro de la Universidad. A través de una experiencia interactiva, los cajeros podrán reconocer el vocabulario clave de los procesos administrativos y financieros, consolidando su aprendizaje de forma práctica con el software.

La implementación del sistema materializa los conceptos esenciales de la asignatura Lógica de Programación, destacando el uso de estructuras repetitivas, condicionales, listas, variables y funciones para dirigir el flujo de ejecución. Adicionalmente, el almacenamiento y la administración del código se realizan mediante GitHub para asegurar un correcto control de versiones.

Implementar mecánicas de juego en la enseñanza constituye un enfoque metodológico que favorece el compromiso laboral, consolida la apropiación de conceptos y potencia las habilidades cognitivas. Bajo esta premisa, el proyecto vincula la programación y el diseño lúdico para ofrecer una experiencia formativa ágil.

REINVENTEMOS
EL FUTURO

Finalmente, el proyecto valida la importancia de la programación como medio para construir aplicaciones enfocadas en el aprendizaje; esto demuestra que soluciones informáticas accesibles pueden transformar radicalmente la asimilación de contenidos de las diferentes capacitaciones.

2. Descripción del problema

En la Universidad existen diferentes procesos o trámites administrativos y financieros, como matriculación, facturación y pagos, cuyo volumen informativo suele confundir a los colaboradores. Al enseñarse tradicionalmente con documentos impresos o largas horas de capacitación e inclusive el envío masivo de correos electrónicos reduce el interés y la retención de los temas impartidos. Ante la falta de recursos interactivos para el autoaprendizaje, este proyecto propone desarrollar un software basado en el juego del Ahorcado, una alternativa lúdica que estimula la memoria, el análisis lógico y la participación autónoma.

3. Relación con los contenidos de la asignatura

El desarrollo del Juego del Ahorcado para el aprendizaje de conceptos de facturación en la Universidad sirvió como un espacio práctico para ejecutar las competencias desarrolladas en la materia de Lógica de Programación. De este modo, se logró unificar y articular las nociones teóricas más relevantes que se abordaron de forma sistemática en el transcurso de este primer módulo.

REINVENTEMOS
EL FUTURO

Como paso inicial, se llevó a cabo el examen detallado de la problemática con el objetivo de detectar un requerimiento específico y proponer una alternativa tecnológica adecuada. Después se elaboraron diagramas de flujo empleando la simbología estandarizada, lo cual simplificó la ilustración visual y secuencial del razonamiento del software de manera previa al comienzo de la fase de creación del código.

En la etapa de construcción del programa se incorporaron condicionales a través de las sentencias if, elif y else, con el objetivo de verificar los caracteres provistos por el jugador. Estas sentencias permitieron calificar el éxito o fallo de cada intento, administrar los estados que definen el triunfo o el fin del juego, y asegurar un correcto tratamiento en la verificación de las entradas del sistema.

De igual manera, se integraron estructuras de control de carácter iterativo mediante la implementación de los bucles for y while. El constructor for, trabajando de la mano con la instrucción range(), sirvió como mecanismo para monitorizar el límite de las seis oportunidades que posee el jugador, además de emplearse para examinar secuencialmente cada uno de los caracteres que conforman el vocablo elegido. Por otro lado, la sentencia while se dispuso específicamente con el fin de examinar la confirmación del participante una vez concluido el juego, garantizando así la opción de volver a iniciar una partida nueva si fuera requerido.

Por otra parte, la organización interna del código hace uso de estructuras mediante la implementación de diccionarios y listas. La función de los diccionarios consiste en centralizar la base de datos de palabras sobre facturación, emparejando de manera directa cada concepto con su correspondiente pista de estudio, mientras que las listas se ocupan de la persistencia temporal de los caracteres alfabéticos que el estudiante va adivinando de forma satisfactoria en el transcurso del juego.

Asimismo, el manejo interno del algoritmo se apoyó en las palabras clave break y continue para modular el flujo y las iteraciones dentro del programa. El uso de la instrucción break cumple la función de romper el bucle activo de manera automática una vez que el alumno logra adivinar la totalidad de la palabra sobre facturación; en contraposición, la sentencia continue se emplea para reiniciar el ciclo desde su comienzo de forma inmediata si es que los sistemas de validación descubren que el dato ingresado es incorrecto o corresponde a una letra que ya se había introducido en turnos anteriores.

En la última etapa del proceso técnico, la infraestructura de desarrollo se apoyó en los entornos de Git y GitHub para la administración de las líneas de código y el control de versiones concurrentes. La adopción de estos recursos facilitó la persistencia de los archivos fuentes en repositorios, la documentación del progreso, y la correcta disposición organizativa de los elementos informáticos indispensables para la construcción del software.

4. Explicación del sistema desarrollado

El software implementado se compone de una dinámica interactiva basada en el tradicional juego del Ahorcado y programada en Python, diseñada con el fin de afianzar el aprendizaje del vocabulario técnico de facturación de la Universidad Internacional del Ecuador (UIDE). La aplicación corre en una terminal o interfaz de consola bajo una metodología simple que vincula el entretenimiento y la educación didáctica, lo que da como resultado un recurso donde el alumno puede ejercitar y medir sus destrezas administrativas mientras juega y se involucra con las funciones del programa.

Como primera acción al inicializar el juego, el algoritmo manda a llamar a la librería random de Python para realizar la escogencia aleatoria de los vocablos y las definiciones que el estudiante deberá resolver, tomándolos de un banco de palabras preestablecido. Gracias a la incorporación de esta herramienta computacional, se evita la anticipación en las partidas, transformando el proceso pedagógico en una actividad interactiva donde el cajero se enfrenta a un desafío distinto cada vez que ingresa a la consola.

Cuando el programa ya ha escogido la palabra técnica que se va a jugar, se la muestra en la pantalla del usuario oculta detrás de una serie de guiones bajos que representan cada una de sus letras, publicando también el enunciado de la pista de facturación para guiar al estudiante. Al mismo tiempo que esto ocurre en la interfaz de la consola, el código activa e inicializa todas las variables internas esenciales para regular el flujo del juego; entre estos elementos de control de datos se encuentran la lista que irá

REINVENTEMOS
EL FUTURO

registrando las letras acertadas y el indicador numérico que fija la cantidad límite de fallos que puede cometer el jugador antes de perder.

La mecánica principal del programa se desenvuelve dentro de un bucle repetitivo *for* que limita la partida a un máximo de seis fallos o turnos gracias al uso de la función *range()*. Mientras esta estructura se mantenga activa en cada vuelta, el código le pedirá al participante que escriba una letra en la terminal y, de inmediato, someterá esa entrada a distintas verificaciones lógicas con el fin de certificar que el juego responda adecuadamente y no se produzcan fallos en el procesamiento del flujo.

El primer filtro del código se encarga de constatar que el cajero haya digitado una sola letra y ningún otro tipo de símbolo o texto largo. Si la entrada del usuario incumple esta regla, la consola muestra un aviso de advertencia y se apoya en el comando *continue* para volver a empezar el bucle inmediatamente, asegurando que no se le descuenta ninguna vida o intento al jugador por este fallo. Una vez superado este control, el programa pasa a examinar si la letra ingresada coincide con alguna que ya se haya intentado antes en la misma partida, impidiendo así que el alumno gaste turnos repitiendo letras y logrando que la navegación por el juego sea mucho más fluida e interactiva.

Cuando el sistema comprueba que la letra digitada es correcta en su formato, pasa a verificar si esta se encuentra dentro de la palabra de facturación mediante el uso de los condicionales *if*, *elif* y *else*. Si el estudiante acierta, el código toma esa letra y la añade a la lista de caracteres adivinados empleando el método *append()*, lo que permite que los guiones bajos de la pantalla se transformen inmediatamente para mostrar las letras descubiertas. En la situación opuesta, es decir, si la letra no pertenece a la palabra, el

REINVENTEMOS
EL FUTURO

juego le avisa al participante sobre el error cometido y salta al próximo intento de la estructura repetitiva, continuando así hasta que el usuario llegue al tope de fallos establecido en el programa.

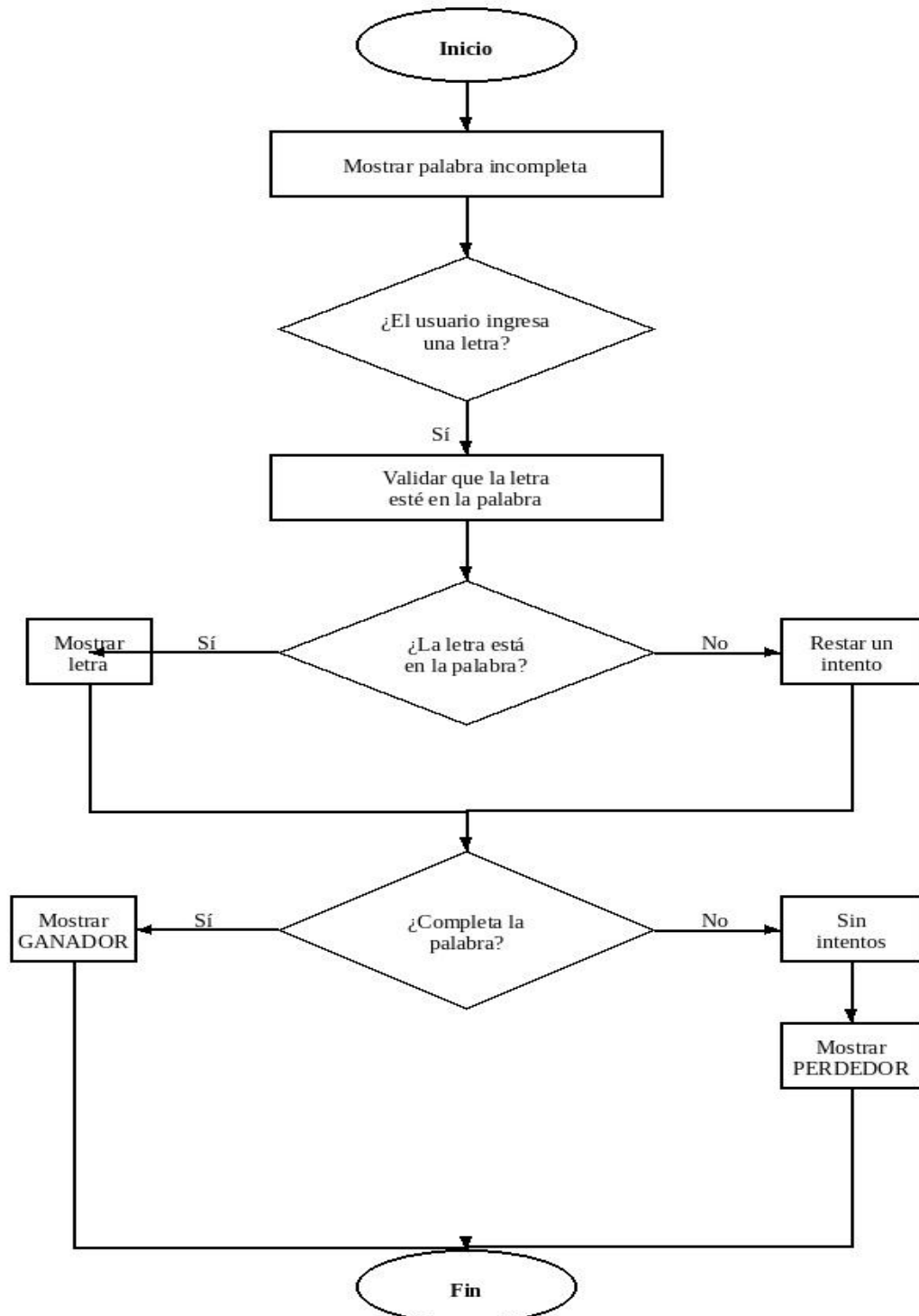
A la par de las validaciones anteriores, el código revisa constantemente en cada vuelta si el estudiante ya consiguió rellenar todos los espacios y descubrir la palabra. Cuando el sistema detecta que no quedan letras ocultas, proyecta en la pantalla de la consola una felicitación y activa el comando break, lo que provoca que el juego rompa el bucle de los seis intentos de inmediato y dé por finalizada la partida con un resultado exitoso para el cajero.

En caso de que el participante agote las seis oportunidades configuradas sin conseguir descifrar el término en su totalidad, la aplicación da por terminada la sesión proyectando una notificación de pérdida del juego. De forma simultánea, el sistema expone explícitamente el vocablo oculto, posibilitando así que el estudiante reconozca con exactitud el término técnico de facturación que formaba parte del juego.

Por último, el programa le presenta al alumno una pregunta en la terminal para saber si quiere volver a jugar otra partida. Para programar este mecanismo, se utiliza un bucle repetitivo while que se encarga de vigilar que la respuesta digitada sea únicamente la letra "S" para indicar que sí o la letra "N" para indicar que no. En el momento en que el usuario confirma que desea seguir aprendiendo, el software reinicia los contadores y las palabras por completo para empezar otra vez; si decide no continuar, el sistema detiene su funcionamiento y se cierra la consola.

A continuación, el desarrollo de los diagramas de flujo:

a. Diagrama lógico del juego



REINVENTEMOS
EL FUTURO

b. Diagrama de funcionamiento en código

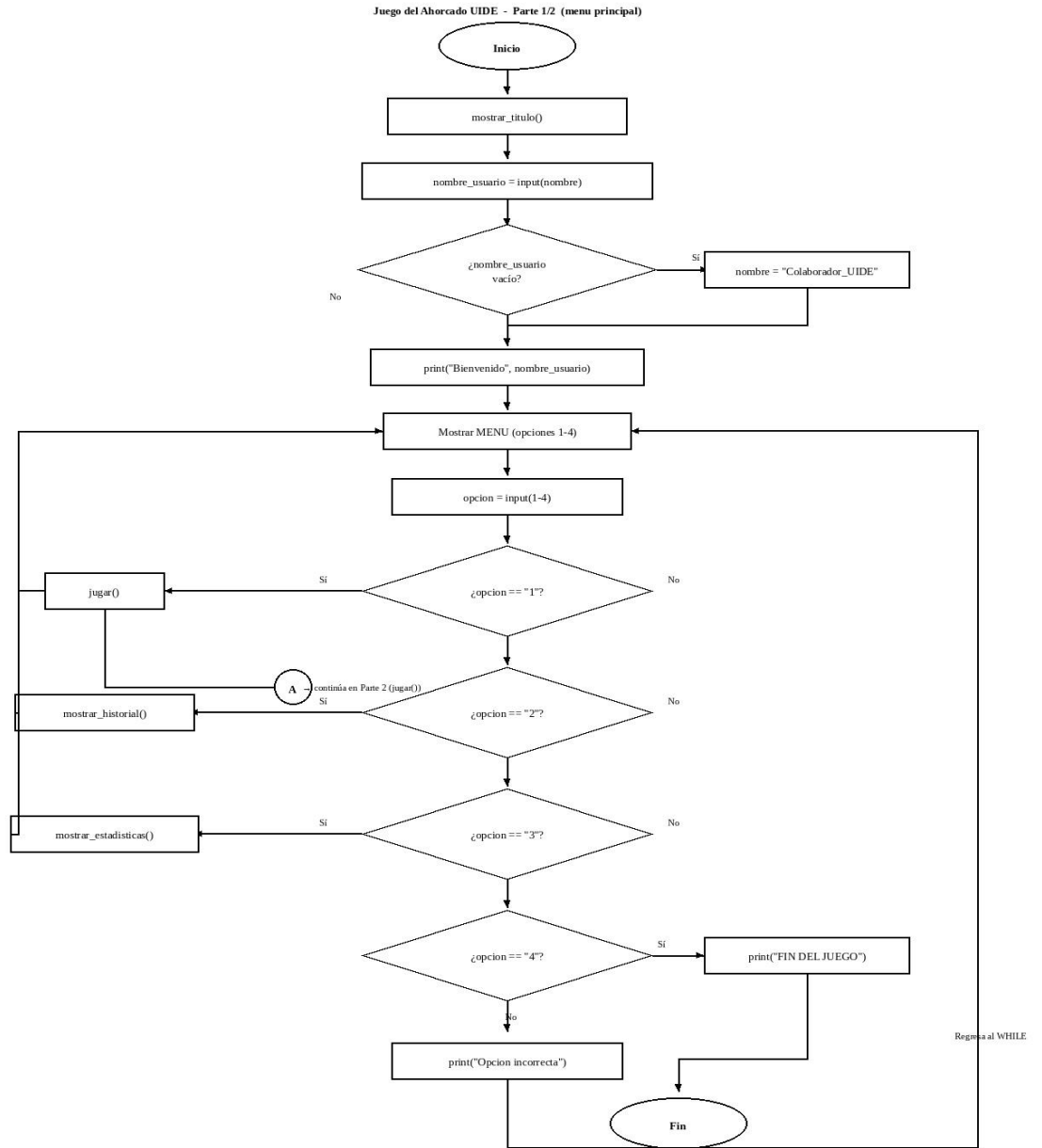
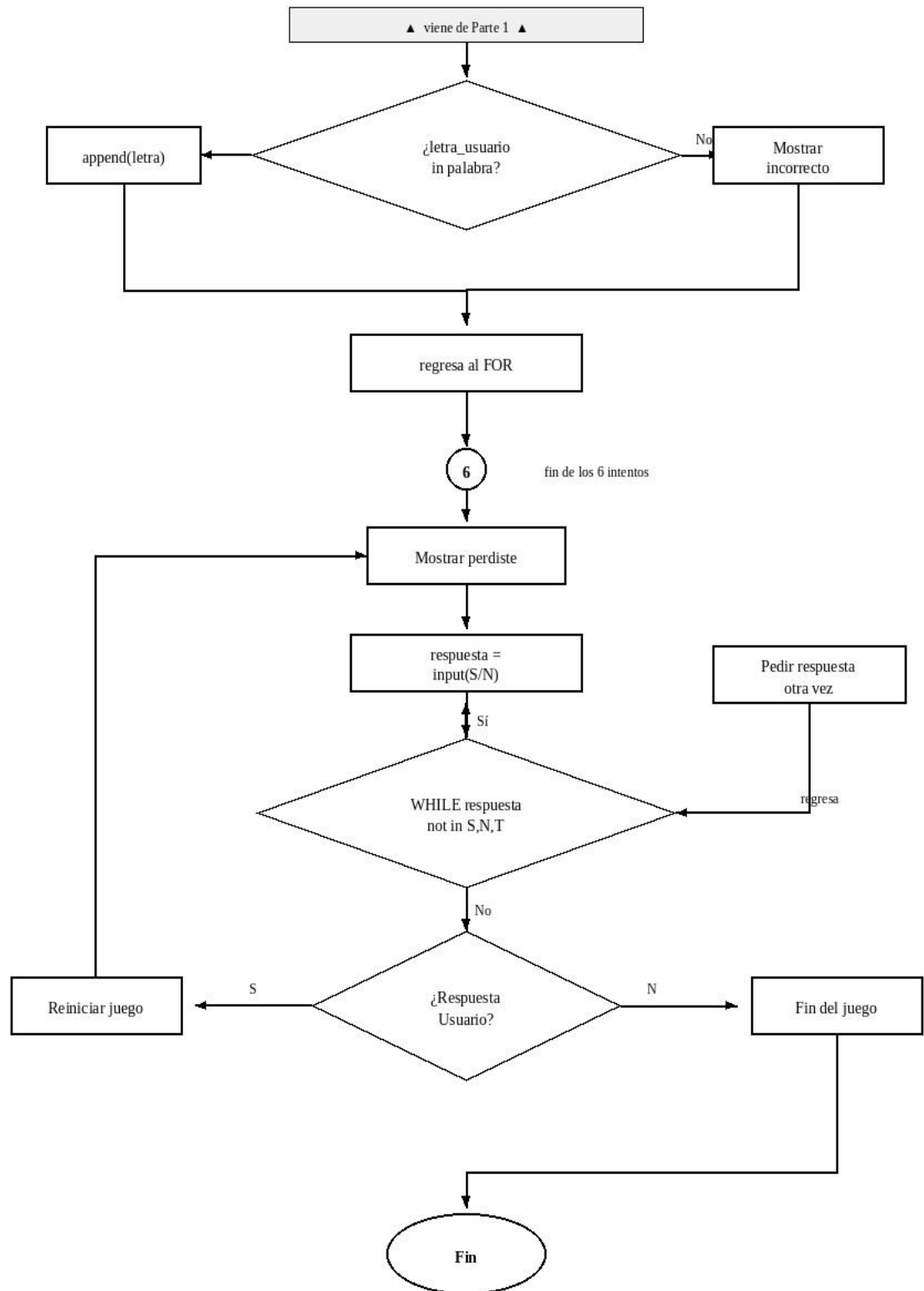


Diagrama código – Juego del Ahorcado (Parte 2/2)



5. Reflexión sobre el impacto de la tecnología

El desarrollo del juego del Ahorcado fue de gran utilidad para la programación del diseño de herramientas de software con un alto valor didáctico y pedagógico. El sistema desarrollado propicia el aprendizaje de los diferentes conceptos en los procesos administrativos y financieros de la Universidad con una metodología que incentiva la proactividad del cajero durante y después de las sesiones de capacitación.

Desde el punto de vista de la ingeniera de software, el diseño de este programa es un ejemplo claro de cómo la aplicación de metodologías de trabajo, el uso de buenas prácticas al escribir código y la adopción de herramientas de control de versiones como GitHub son pasos cruciales para lograr aplicaciones bien organizadas, fáciles de actualizar con el tiempo y preparadas para expandirse. Estas habilidades técnicas y organizativas son fundamentales para cualquier profesional del área, ya que son las herramientas que garantizan el desarrollo de sistemas estables, seguros y con altos estándares de calidad exigidos en la actualidad.

En este mismo sentido, construir e implementar este programa nos ayudó a asimilar que los avances tecnológicos no sirven únicamente para hacer que los procesos sean automáticos, sino que tienen un enorme potencial para enriquecer los métodos de enseñanza por medio de recursos didácticos creativos, accesibles y muy interactivos. El diseño de este tipo de herramientas educativas ayuda a expandir la creatividad de los alumnos, afianza las bases del razonamiento lógico y los motiva a estudiar por su propia cuenta de forma independiente, cualidades que hoy en día son vitales para garantizar la excelente preparación de cualquier estudiante en su camino a convertirse en profesional.

REINVENTEMOS
EL FUTURO

Finalmente, este proyecto permitió evidenciar que los conocimientos adquiridos en la asignatura de Lógica de Programación tienen una aplicación práctica en la creación de soluciones reales. La combinación de algoritmos, estructuras de control, organización del código y herramientas de desarrollo demuestra que la programación constituye una disciplina clave para responder a las necesidades de la sociedad actual. En consecuencia, la tecnología representa una oportunidad para innovar en los procesos educativos, mejorar la experiencia de aprendizaje y contribuir al desarrollo de competencias digitales que serán indispensables en el ámbito académico y profesional.

REINVENTEMOS
EL FUTURO